

Project 8: Timer-Based Digital Clock and Task Scheduler System

ATmega32 System Function Covered:

Time delay and time measurement system.

Project Theme:

Timer programming, timekeeping, periodic task scheduling, embedded timing control, and real-time system behavior.

Problem Statement

Many computer engineering systems need accurate timing without stopping the main program. Examples include digital clocks, periodic sensor sampling, display refreshing, communication timeouts, and scheduled control tasks. Software delay loops are simple, but they waste processor time and become inaccurate when the program grows. The ATmega32 timers allow the microcontroller to generate regular timing events using hardware timer resources.

This project introduces students to timer-based timekeeping and periodic task scheduling using the ATmega32.

Project Objective

To design and implement a **timer-based digital clock and task scheduler system** using an ATmega32 timer interrupt. The system should maintain time in seconds, minutes, and hours while also executing periodic tasks such as LED blinking and sensor sampling indication.

Expected System Operation

The system should use a hardware timer to generate a regular time base, such as 1 ms, 10 ms, or 1 second. From this time base, the firmware should update a digital clock and run scheduled tasks.

System Condition	Expected Output
System powered ON	Clock starts from 00:00:00
Timer interrupt occurs	Internal time counter updates
Every 1 second	Seconds value increases
Every 2 seconds	Status LED toggles
Every 5 seconds	Sampling indicator LED flashes
Set button pressed	Time-setting mode is activated
Reset button pressed	Clock resets to 00:00:00

Example display format:

TIME: 00:05:27

TASK: SENSOR SAMPLE

Main Hardware Components

Component	Quantity	Purpose
ATmega32 microcontroller	1	Main controller
LCD module or serial terminal interface	1	Displays clock time and task status
Push buttons	2 or 3	Time setting and reset input
LEDs	2	Status and scheduled-task indication
Current-limiting resistors	As required	LED protection
Reset push button	1	Manual microcontroller reset
Crystal oscillator	1	Stable microcontroller clock
Capacitors	As required	Clock and power filtering
5 V power supply	1	System power

ATmega32 Pins / Functions Used

ATmega32 Resource / Pin	Function	Connected Device
Timer/Counter1	16-bit timing resource	Generates accurate periodic interrupt
PB0	Digital output	Status LED
PB1	Digital output	Sampling/task indicator LED
PA0	Digital input	Set button
PA1	Digital input	Increment button
PA2	Digital input	Reset clock button

Optional PD1 / TXD	USART transmit	Sends clock output to serial terminal
Optional LCD GPIO pins	Digital output	Displays clock and task messages

The main system function is:

Hardware Timer → Periodic Interrupt → Timekeeping and Scheduled Tasks}
Hardware Timer → Periodic Interrupt → Timekeeping and Scheduled Tasks}

KiCad Schematic Design Requirement

Students must design the complete circuit schematic in KiCad.

The schematic should include:

Schematic Part	Requirement
ATmega32	Clearly show GPIO pins used for buttons, LEDs, and display/serial output
Timer resource	Timer is internal, but the schematic should document which timer is used in the notes
Clock circuit	Include external crystal and capacitors for stable timing
LEDs	Connect status and task LEDs through current-limiting resistors
Buttons	Connect set, increment, and reset buttons with stable pull-up or pull-down configuration
Display or serial header	Include LCD interface or USART header if used
Reset circuit	Include reset push button and pull-up resistor
Power supply	Include VCC, GND, AVCC, and decoupling capacitors
Net labels	Use labels such as SET_BTN, INC_BTN, CLOCK_RESET, LED_STATUS, LED_TASK, USART_TXD

KiCad PCB Design Requirement

Students must convert the schematic into a PCB layout using KiCad.

The PCB should include:

PCB Requirement	Description
Clock placement	Place crystal and capacitors close to the ATmega32
Button placement	Place time-setting buttons where users can operate them easily
LED placement	Place LEDs where they are visible during testing
Display connector	Place LCD or serial connector in a convenient location
Power filtering	Place decoupling capacitors close to the microcontroller
Silkscreen labels	Label buttons, LEDs, display/serial connector, power, and reset
Design checks	Run ERC and DRC
Output files	Generate Gerber files

MPLAB for VS Code Coding Requirement

Students must write firmware using **MPLAB for VS Code**.

The firmware should perform the following tasks:

Firmware Task	Description
Timer initialization	Configure Timer/Counter1 for periodic interrupt
Prescaler selection	Choose timer prescaler based on system clock
Compare value calculation	Calculate compare-match value for selected time base
Interrupt configuration	Enable timer compare-match interrupt
Timekeeping logic	Update milliseconds, seconds, minutes, and hours
Task scheduler logic	Run tasks at selected intervals
Button handling	Set, increment, or reset time
Display update	Show current time and task status
Non-blocking program structure	Avoid long delay loops in main program

For timer compare operation, the compare value can be calculated as:

$$OCR1A = \frac{F_{CPU}}{\text{Prescaler} \times f_{\text{interrupt}}} - 11$$

$$OCR1A = \frac{F_{CPU}}{\text{Prescaler} \times f_{\text{interrupt}}} - 11$$

For example, if:

$$F_{CPU} = 8, MHz$$

$$F_{CPU} = 8, MHz$$

$$\text{Prescaler} = 1024$$

$$\text{Prescaler} = 1024$$

$$f_{\text{interrupt}} = 1, Hz$$

$$f_{\text{interrupt}} = 1, Hz$$

then:

$$OCR1A = \frac{8,000,000}{1024 \times 1} - 1$$

$$OCR1A = \frac{8,000,000}{1024 \times 1} - 1$$

$$OCR1A \approx 7812$$

$$OCR1A \approx 7812$$

This means Timer1 can generate one interrupt per second using compare match mode.

SimulIDE Simulation Requirement

Students must build and test the timer-based system in **SimulIDE**.

The simulation should show:

Simulation Test	Expected Observation
System startup	Clock displays 00:00:00
One-second timing	Seconds value increases once per second

Status task	Status LED toggles every 2 seconds
Sampling task	Task LED flashes every 5 seconds
Set button	Time-setting mode activates
Reset button	Clock returns to 00:00:00

Students should capture screenshots showing clock operation, LED scheduling, and button response.

GitHub Repository Requirement

Students must create a GitHub repository and share it with the instructor.

Suggested repository name:

ATmega32_Timer_Digital_Clock_Scheduler

Suggested structure:

ATmega32_Timer_Digital_Clock_Scheduler/

```
|
|
|— README.md
|
|
|— firmware/
|  |— src/
|  |— include/
|  └─ hex/
|
|
|— kicad/
|  |— schematic/
|  |— pcb/
|  └─ gerber/
|
|
|— simulide/
|  |— circuit/
|  └─ screenshots/
```

```
|
|
|└─ documentation/
|
|  |└─ block_diagram.png
|
|  |└─ flowchart.png
|
|  |└─ timer_configuration.md
|
|  |└─ compare_match_calculation.md
|
|  |└─ pin_mapping_table.md
|
|  └─ test_results.md
|
|
└─ media/
    └─ demonstration_video_link.txt
```

Recommended commit examples:

Created timer scheduler project repository

Added Timer1 compare-match firmware

Added digital clock timekeeping logic

Added periodic task scheduler logic

Added KiCad schematic for timer project

Added SimulIDE simulation screenshots

Added PCB layout and Gerber files

Added timer calculation and test documentation

Software Requirements

Software Requirement	Description
Timer mode selection	Use Timer1 CTC mode or equivalent timer mode
Compare-match setup	Configure compare value for fixed interrupt interval
Interrupt service routine	Update timing counters inside ISR
Clock logic	Convert seconds into hours, minutes, and seconds
Scheduler flags	Use flags for periodic tasks instead of long delay loops

Button debounce	Avoid false button readings
Display routine	Update LCD or serial output clearly
Modular coding	Separate timer, display, button, and scheduler functions

A suitable software structure is:

```
main()
{
    initialize_gpio();
    initialize_timer1();
    initialize_display();

    while(1)
    {
        check_buttons();
        update_display_if_needed();
        run_scheduled_tasks();
    }
}
```

```
ISR(TIMER1_COMPA_vect)
{
    update_time_base();
    set_scheduler_flags();
}
```

Student Design Tasks

Task	Description
1	Draw the system block diagram
2	Select timer resource and time base

3	Calculate timer prescaler and compare value
4	Prepare the ATmega32 pin mapping table
5	Design the schematic in KiCad
6	Design the PCB layout in KiCad
7	Write timer firmware in MPLAB for VS Code
8	Implement timer interrupt and scheduler logic
9	Simulate the system in SimulIDE
10	Test clock accuracy, LED scheduling, and button response
11	Upload all files to GitHub
12	Present the working system to the instructor

Expected Output / Demonstration

Demonstration Item	Expected Evidence
Timer interrupt operation	Clock updates at correct interval
Digital clock	Time is displayed as hours, minutes, and seconds
Scheduled LED task	Status LED toggles every 2 seconds
Scheduled sampling task	Task LED flashes every 5 seconds
Button control	Time can be set or reset
KiCad schematic	Complete schematic file and screenshot
KiCad PCB	PCB layout screenshot and Gerber files
MPLAB for VS Code project	Source code and HEX file
SimulIDE simulation	Working simulation and screenshots
GitHub submission	Repository link with organized files

Assessment Focus

Assessment Area	What to Assess
Timer understanding	Correct use of hardware timer for time base generation
Compare-match calculation	Correct prescaler and compare-value computation

Interrupt programming	Correct timer interrupt setup and ISR structure
Scheduler design	Proper non-blocking periodic task execution
Firmware quality	Modular code, readable logic, and meaningful comments
Simulation quality	SimulIDE behavior matches expected timing operation
Schematic design	Proper clock, reset, display, button, LED, and power wiring
PCB design	Good clock placement, labels, and design-rule compliance
GitHub usage	Organized repository and meaningful commits
Documentation	Includes timer configuration, calculations, flowchart, and test results

Extension Task

Students can extend the project by adding **multiple software timers**. For example, one task runs every 500 ms, another every 2 seconds, and another every 10 seconds. Students should implement this using timer-generated scheduler flags, not blocking delay loops.